



PRO SKILL PACKS

The Free Prompt Book

21 tested prompts for everyday work, grouped by job.
Copy one, paste it into a chat, add your material.

Works in ChatGPT, Claude, Gemini or any chatbot. Nothing to install.

proskillpacks.github.io



How to use this book

Every prompt here is plain text. It works in ChatGPT, Claude, Gemini or any chatbot, in a new chat, with nothing to install.

Three steps

1. Pick the job. Read the short note on the prompt's page: what it does and what to have ready.
2. Copy everything in the grey box and paste it into a new chat. Then paste your own material under it: the text, the table, the page or the list the note asks for.
3. Read the answer before you use it. The prompts are written to say what they could not check and to leave gaps marked instead of guessed. Fill those gaps yourself.

Good to know

- A chatbot can only work from what you paste. If a prompt asks for a web page, paste the text of the page unless your chatbot can open links.
- Use your assistant's strongest model. In our tests a small, fast model added facts that were not in the material.
- These are tools, not guarantees. Nothing here is legal, tax or medical advice.

In this book

E-commerce

Agent-Ready Quick Check
Chargeback Evidence Checklist
Product Title Cleaner
Review Reply Drafter
Shopify Alt Text Writer
Shopify Policy Checker

Marketing and SEO

AI search readiness check
Search intent page brief

Freelancers

Late Payment Chaser (UK)

Writing

Fact-Claim Checker

Sales

Cold email checker

Data and spreadsheets

CSV profile and sanity check

Careers

Job post decoder



Customer support

Angry customer reply check

Finance and admin

Invoice checker

Developers

Accessibility quick audit

Changelog from Commits

PR description and review prep

README First-Run Check

Skill Portability Check

Test Gap Finder

**E-COMMERCE**

Agent-Ready Quick Check

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free Agent Skill that checks, in about 30 seconds, how well AI shopping agents (ChatGPT, Perplexity, Gemini and Google AI Mode, Copilot, Claude) can read your Shopify store, and compares your store with 99 other Shopify stores.

What to paste

- Open <https://yourstore.com/robots.txt> and copy the whole page.
- Open <https://yourstore.com/llms.txt> and note whether it loads (a page of text) or gives "page not found".
- Open one typical product page, view its source (Ctrl+U, or Cmd+Option+U on a Mac), search for `application/ld+json` and copy each block that follows it, from `{` to the matching `}`.
- Open the same product with `.json` added to its address (<https://yourstore.com/products/<handle>.json>) and copy the whole page. It holds the description and the image alt text.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are checking how well AI shopping agents can read a Shopify store, from what is pasted below. Work only from it. Never estimate a result you can't see in the pasted text: mark that check "not measured".

The six checks (pass rules):

1. AI agents allowed in + llms.txt. Pass if robots.txt does not block any of OAI-SearchBot, ChatGPT-User, PerplexityBot, Perplexity-User, Claude-SearchBot, Claude-User, Googlebot or Bingbot from product pages (a rule under "User-agent: *" counts for agents not named separately; blocks on /cart, /checkout, /admin, /orders, /account don't count), and llms.txt loads. Agents not in that list (for example Amazonbot) don't affect the result; mention them in one line if blocked.
2. Shipping and returns in structured data. Pass if a Product or Offer in the JSON-LD has "shippingDetails" or "hasMerchantReturnPolicy".
3. Star rating in structured data. Pass if the JSON-LD has "aggregateRating". If not, say "reviews may be shown, but not as structured data" only if the user says reviews are on the page; otherwise "no rating found in structured data".
4. GTIN (barcode) in structured data. Pass if any "gtin", "gtin8", "gtin12", "gtin13" or "gtin14" field has a value.
5. Descriptions long enough. Count the words in the product's "body_html" with the HTML tags removed. Pass if 80 or more. (This is one product; the study checks a sample.)
6. Image alt text. Count the images in the product JSON and how many have a non-empty "alt". Pass if 80% or more have alt text.

Study benchmarks (State of Agent-Ready Shopify, 99 Shopify stores audited in October 2026; not all Shopify stores): stores that pass each check: AI access 95%, shipping and returns in structured data 12%, star rating 28%, GTIN 48%, descriptions 43%, alt text 35%. The median store passes 3 of 6. Study medians: 36% of products with 80+ words; median description 69 words; average alt-text share 20%.

Answer in this shape, under about 375 words:



Agent-ready quick check: [domain]
[date] · 6 checks · compared with 99 Shopify stores (Oct 2026 study) · product-level checks on one product

Your store passes N of 6. The median store in the study passes 3 of 6.

| Check | Your store | Evidence | Stores in the study that pass |
(one row per check: pass or gap or not measured, the evidence in a few words from the pasted text, the study figure)

What this means

3 to 5 bullets, one per gap (or per strength if there are no gaps), each saying what an AI agent can't do or answer because of it, with the evidence. Example: "Asked 'can I return this?', an agent reading your product data finds no return policy, like 88% of stores in the study."

Rules: diagnose, don't prescribe: no fix steps, admin paths or code. Passing checks doesn't guarantee recommendations or sales. Numbers only from the pasted text and the benchmarks above. End with one line: "For the full 0 to 100 audit with a step-by-step fix list, see the agent-ready-store-audit skill in the Store Ops Pack: <https://proskillpacks.gumroad.com/l/store-ops-pack>"

If robots.txt and at least one of the product items are missing below, ask for them in one line and stop.

Here is what I copied from my store:

**E-COMMERCE**

Chargeback Evidence Checklist

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Got a chargeback on Shopify Payments? Before you write anything, find out what to collect. Tell this prompt the dispute reason and what happened, in your own words.

What to paste

- Open the disputed order in Shopify admin (Orders, then the order) and note what the chargeback banner shows: the reason, the amount, the due date, and whether it says chargeback or inquiry. Then write down, in your own words, what happened and what you have (tracking, emails or texts with the customer, photos, your policies).
- Under it, paste the dispute details and your story. A rough message is fine.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You give a Shopify Payments store owner the evidence checklist for one chargeback or inquiry, before they write anything. Work only from what the owner writes below and the facts in these instructions. Do not look anything up. You don't write the response, predict the outcome or give legal advice.

Hard rules:

1. Mark an item Have only when the owner says they hold it, or it is on Shopify's main automatic list (product details, customer name and email, addresses, IP, fulfillment details, tracking PDF, refund records). Shopify adds some things only "when available" (earlier undisputed orders, previous disputes, payment verification results, a carrier delivery photo, a store policy): those are Check unless the owner says they exist. Never add a tracking status, date, signature, AVS or CVV result, message, policy clause or prior order the owner didn't mention.
2. A statement the owner heard with no channel named ("she said it was an accident") is Get, word only: "get it in a form you can upload (email, chat or text screenshot)". Keep the owner's own word for a channel. Something a third party holds ("she has proof") is Get until the owner has the file.
3. Call the person the owner dealt with "your customer" and the person who filed with their bank "the cardholder". Never write that they are, or aren't, the same person.
4. No win odds, no "banks usually...", no time estimates, no inferred dates or years. Don't guess the store's country from a carrier, a card brand or where goods are made.
5. If the reason isn't given, say so first, point to the chargeback banner, pick the reason the facts point to (name those facts), and add up to 3 lines on what changes for another reason.

Shopify's evidence lists (from Shopify's Help Center, October 2026):

- Fraudulent (and Unrecognized): AVS and CVV results, 3D Secure record, IP and device data consistent with earlier orders, earlier undisputed orders from the same customer, delivery confirmation to the verified address, messages where the customer acknowledges the order. Delivery proof alone doesn't answer fraud. Unrecognized also: the statement descriptor and the order confirmation email. Some Shop Pay orders are covered by Shopify Protect: check the order. Visa Compelling Evidence 3.0 is a condition to check (two earlier undisputed payments on the same card 120 to 365 days before, matching IP and device, or one of those plus shipping address, email or account ID), never something to assume.
- Product not received, physical: tracking (carrier, number, status), delivery confirmation (ideally signature or photo), the shipping address matching checkout, carrier delivery notifications. Digital: access logs, the delivery email with the link or key, usage timestamps. Services: booking records, completion proof, customer acknowledgment. Leave the return policy out.



- Product unacceptable: the listing as it was at purchase, what was ordered and shipped and when, pre-shipment photos, quality records, messages about the issue and the resolution offered, the return policy and that the customer agreed to it. If the customer never contacted the store: evidence of that, plus the store's contact and resolution process.
- Credit not processed: the refund record (date, amount, confirmation), a bank or processor statement matching it, the refund policy agreed at checkout, messages about the refund, return tracking if a return is involved. If a refund is owed and wasn't paid, accepting is the match.
- Subscription canceled: the subscription terms, the cancellation policy, cancellation records, usage after the charge, renewal reminders actually sent, messages about the subscription.
- Duplicate: transaction records, an order comparison (numbers, timestamps, items; on the order timeline), receipts for each, the refund if a real duplicate was refunded. Authorization holds aren't charges.
- General: itemized receipt, the final amount shown at checkout, pricing policy, exchange rate used, pre-purchase messages about price.
- Customer says they withdrew the dispute: it stays open until the bank closes it, so respond before the due date anyway. Get the bank's withdrawal letter from the customer: on bank letterhead, with the case number, the original charge date and amount, the store name, confirming the chargeback is cancelled, the funds re-debited from the customer's account and returned to the merchant. A bank statement screenshot isn't enough.

Where things are in Shopify: the dispute, reason, amount and due date are on Orders, then the order, then the chargeback banner and chargeback details section. Evidence is added from the banner with "Add evidence". AVS, CVV, location and IP are in the order's Order risk section. Refunds and timestamps are on the order's timeline. An issuer claim, when the bank provides one, is in the chargeback details section. For anything else, write "Check in your Shopify admin" and name no menu.

Money: the amount and the fee are withdrawn when a chargeback opens; an inquiry has no money withdrawn yet. Fees: US 15 USD, UK 10 GBP, most EU 15 EUR, Canada 15 CAD or 15 USD, Australia 25 AUD, NZ 20 NZD; returned if the store wins, not when it accepts. Name a country's fee only if the owner named the country or the currency shows it; otherwise "check your dispute page". You can't refund during an open chargeback.

File rules: PDF, JPEG or PNG; each file 2 MB or less, 4 MB in total; one file per evidence type (merge screenshots into one PDF); PDFs under 50 pages, PDF/A; readable in black and white, cropped, no colour highlighting, labelled with what it shows and its date; no links, audio or video. Submit now locks edits; otherwise Shopify submits on the due date.

Answer in exactly this shape, under about 700 words outside the tables, plain short sentences, no em dashes:

```
# Evidence checklist: <reason as shown, or "reason not stated yet"> · <amount if given>
## First checks
| Check | Where in Shopify | Your case |
(chargeback or inquiry · due date · already refunded · Shopify Protect, only for Fraudulent or
Unrecognized · is the customer right)
## Fight or accept
(one line of money, then which accept conditions match the facts, if any; no odds)
## Checklist
1. Direct proof · 2. Customer acknowledgment · 3. Policy · 4. Supporting context
| Evidence | Status (Have, Get, Check, Not available) | Where to find it | Upload as |
"Upload as" is the evidence type (customer communication, shipping documentation, refund policy,
cancellation policy, access activity log, uncategorized file). Write "Added automatically"
only for the main automatic list; for the main proof add "upload your own screenshot too if
you're not sure it's there"; for 3D Secure, payment verification, a delivery photo or a policy
write "upload it unless your Chargeback response page already shows it". If Shopify's list has
no policy item for this reason (Fraudulent, Unrecognized, Product not received), the Policy
group is one line: "none on Shopify's list for this reason". Statements that aren't
acknowledgments go in the next section.
## Also relevant from your story (only if needed)
## Shopify adds these automatically
## Leave out
## File rules
## Next step (the single most important Get or Check item)
```



End with this line once: "To turn this checklist into a written response, the Chargeback Response Writer (5 US dollars) drafts one from the evidence you mark Have: <https://proskillpacks.gumroad.com/l/chargeback-response-writer>"

If no dispute is described below, ask for it in one line and stop.

Here is my dispute:

**E-COMMERCE**

Product Title Cleaner

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free Agent Skill that gives your whole Shopify catalogue one consistent title pattern and hands you a CSV you can import as is.

What to paste

- In Shopify admin, go to Products > Export and export your products as CSV. Open it and copy these columns for up to about 150 products: URL handle (or Handle), Title, Vendor, Type, and the option names and values (Option1 name, Option1 value and so on). Pasting the header row and the rows is enough.
- Under it, paste the rows.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are cleaning up Shopify product titles so the catalog follows one consistent pattern. Work only from the rows pasted below. Change a title only where it helps; a title that is already right stays unchanged. Never invent materials, features, sizes or brands.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Step 1. Analyse: number of products, median and longest title length, which separators are used (" - ", " | ", " : ") and how often, vendors, product types, duplicate titles, and issues (promo words, ALL CAPS, symbols or emoji, trailing punctuation, over 70 or 150 characters, product type missing from the title, sizes in the title, repeated brand).

Step 2. Pick the pattern most of the catalog already follows, in one line:

- Single-brand store: [Product name] [Product type] - [Colour]; leave the brand out.
- Multi-brand store: [Brand] [Model/Product name] [Product type] - [Colour].
- Consumables: [Product name] [Type] ([Size/Count]) when the size is fixed and not a variant.

Rules:

1. Variants stay out of the title: if Size (or another option) is a product option, don't put it in the title. Colour stays only when each colour is a separate product.
2. Remove promo and status words (NEW, SALE, Last Call, Free Shipping, Best Seller, emoji, "!!!"); suggest a tag or collection instead. If removing a word would make two titles identical, keep it and flag the pair.
3. Title Case for product names (small words like "and", "of", "with" lowercase unless first). Keep the brand's own styling, trademarks (™ ®), acronyms and model numbers.
4. Add the product type only if a shopper couldn't tell what the item is, using words from the store's own Type or title, and not past 70 characters unless the item is unclear without it.
5. One separator across the catalog (the store's most used). No trailing punctuation or double spaces.
6. Aim for 25 to 70 characters; never over 150.
7. Skip non-merchandise: gift cards, placeholder or "(Copy)" products, tests, shipping protection. List them as skipped.

Step 3. Write the new title for every product. Then check: no two products end up with the same title; nothing over 150 characters.

Answer in this shape:



1. The pattern in one line, and why (with a count from the data).
 2. Counts: products checked, changed, unchanged, skipped (with reasons), titles over 70 characters before and after.
 3. A before/after table of 8 to 12 typical changes, covering each kind of fix.
 4. Flags that need a person: duplicates, brand-styling questions, unclear product types.
 5. The import CSV in a code block, header "URL handle,Title", changed rows only. Quote any title that contains a comma.
 6. How to import: back up first (Products > Export > All products); Products > Import, tick "Overwrite products with matching handles"; columns not in the file keep their values; URL handles don't change; check two or three products afterwards.
- Keep the tone neutral. Don't promise traffic or ranking gains.

If no product rows are pasted below, ask for them in one line and stop.

Here are my products:

**E-COMMERCE**

Review Reply Drafter

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free Agent Skill that drafts replies to your store's reviews. Paste in reviews from Judge.me, Shopify Product Reviews, Yotpo, Okendo, Google or Trustpilot, and you get back a reply for each one, ready to paste.

What to paste

- Copy the reviews you want to answer from Judge.me, Shopify Product Reviews, Yotpo, Okendo, Loox, Google or Trustpilot: name, stars, date, product and text for each.
- Under it, paste the reviews. Add your support email and any remedy you allow (for example "replacement if broken within 30 days"), if you want them used.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are drafting public replies to a Shopify store's customer reviews. Each reply should read as if the owner wrote it to that one customer. Work only from the reviews and details pasted below. Never invent facts (policies, restock dates, causes, "a one-off"); use a placeholder instead.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Defaults: voice "we", no sign-off name, [support email], and no approved remedies. If the owner gives a support email or other value, write it literally in every reply.

Steps:

1. Number the reviews 1 to N in the order given.
2. Classify each: praise, mixed, product-quality, not-received or late, damaged or wrong item, preference (scent, fit, taste), price, subscription, stock-out, service-complaint, rating-mismatch (e.g. 1 star with glowing text), not-a-review (a question), duplicate.
3. Flag where it applies. A flag replaces the normal reply with a holding reply.
 - SAFETY-health: rash, burn, allergic reaction, headache, sick, injury, harm to a child or pet.
 - SAFETY-contamination: bugs, mould, hair in product, opened, used, tampered, expired.
 - LEGAL-threat: lawyer, sue, small claims, FTC, BBB, trading standards, "report you".
 - MONEY-dispute: chargeback, dispute, "went to my bank", card or PayPal claim.
 - ACCUSATION: scam, fraud, fake, "on purpose".
 - PERSONAL-data: phone, email, address, order number or a staff member's full name in the review.
 - ABUSE: slurs, threats, harassment.Strong words alone ("worst", "never again", "overpriced") are not flags.
4. Draft, then check every reply against the rules and rewrite any that fails.

Reply rules:

- Voice "we" throughout. Use the reviewer's first name if shown; none for "Anonymous".
- Echo one specific detail from their review. No two replies open with the same three words; no sentence repeated in the batch.
- Length: happy replies 15 to 45 words; unhappy or flagged 35 to 80.
- Never use: "We value your feedback", "Your satisfaction is our top priority", "We apologize for any inconvenience", "We strive to", "Thank you for taking the time", "We're sorry to hear about your experience", "Please don't hesitate", "valued customer".



- Happy: thank them for the specific thing; no upsell. Rating-mismatch: reply as to praise and note it for the owner; never ask publicly for a rating change.
- Unhappy: acknowledge the problem in their words, apologise for the experience (not the cause), then one next step in a private channel: "Email [support email] with your order number and we'll [REMEDY: replacement / refund / store credit; owner to choose]." Never ask for an order number, address or photos in public. No liability ("our fault", "defect", "known issue"); for health complaints write "didn't agree with you", not the cause. No refund, credit or discount unless the owner approved it, and only within its stated scope. Don't argue or blame the customer, carrier or app.
- Flagged: a holding reply of 35 to 60 words that names what they reported neutrally, says you want to look into it (different wording for each flagged review), and asks them to email [support email] with their order number so a member of the team can follow up. For SAFETY-health add: "If you're still having symptoms, please stop using it and check with a medical professional." No cause, no remedy, not even the placeholder.
- Google: under about 500 characters. Trustpilot: never ask for a rating change or offer anything for editing it.

Placeholders, exactly as written and always inside a sentence: [support email] · [REMEDY: replacement / refund / store credit; owner to choose] · [exchange/return option] · [restock date] · [direct contact: name/email of a person] · [answer: ...]

Answer in this shape:

- 1 to 3 reviews: for each, "### #N · Name · stars · Product", then "Type: ... · Flag: ...", then the reply.
- 4 or more: one table, | # | Reviewer | Stars | Product | Type | Flag | Reply (ready to paste) |, each reply one paragraph.
- Then "Needs a human (N)": for each flagged review, the flag, severity (low, medium, high), what to check (order, photos, batch, similar reviews) and who should see it.
- Then "Other notes" (rating mismatches, trends) and "Placeholders to fill".

If no reviews are pasted below, ask for them in one line and stop.

Here are the reviews:

**E-COMMERCE**

Shopify Alt Text Writer

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free Agent Skill that writes alt text for your Shopify product images. It **looks at each photo** and describes what's actually in it: the angle, the detail, the colour, the model. Then it gives you a CSV you can import into Shopify.

What to paste

- For each product, open `https://yourstore.com/products/<product-handle>.json` in your browser (the handle is the last part of the product's address) and copy the whole page. Or paste the product's title, options, and for each image its position, file address and current alt text.
- If your chatbot accepts images, also upload the product photos, in the same order. It writes better alt text when it can see them.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are writing alt text for a Shopify store's product images. Good alt text tells a screen-reader user what the photo shows and names the product the way a shopper would search for it. Work only from the product data and photos given below.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Rules:

- Pattern: what the photo shows, with the product name worked in once, naturally. Example: "Women's Classic Flip Flop in Dusty Pink, seen from the left side on a white background".
- 60 to 125 characters; never over 125.
- Name the product as the title does, plus the variant colour or material if the image is linked to one or clearly shows one.
- Say what is specific to each photo: angle (side, back, sole, top-down), a close-up of a feature, worn by a model, in use, packaging, or a size chart (with its key facts if short). Every image of a product gets a different alt text; no "alternate view 3".
- If you can't see a photo, use the file name and position as hints (LEFT, BACK, SOLE, TD for top-down, 3Q for three-quarter view, DETAIL) and say in your summary that the text was not checked against the photos. Never describe colours, backgrounds or details you can't know.
- Don't start with "Image of", "Photo of" or "Picture of". No keyword stuffing, prices, promo text, emoji or straight double quotes (use ' if needed).
- Keep existing alt text that is already specific, accurate and under 125 characters. Replace it if it is empty, a file name, just the product title repeated, or wrong.
- Don't claim SEO ranking gains or legal compliance; alt text helps accessibility and image search.

Answer in this shape:

1. One line: "N images across M products: X written from the photos, Y from product data only, Z existing kept."
2. A table: image (handle and position), old alt text, new alt text, character count.
3. The import CSV in a code block, with exactly this header: URL handle,Title,Product image URL, Image position,Image alt text. One row per image, every image of each product in its current order; Title only on each product's first row.
4. How to apply it safely:



- Back up first: Products > Export > All products (CSV).
- Test on one product: keep only its rows, then Products > Import with "Overwrite products with matching handles", and check its images and alt text in the admin.
- Then import the full file. Shopify re-downloads image addresses during an import, so don't run it twice by mistake.
- Products where photos are linked to variants (for example one photo per colour): an import may reset which photo shows for each colour, so for those, paste the alt text by hand (Products > open the product > click the image > Add alt text).

If no product data is pasted below, ask for it in one line and stop.

Here is my product data:

Limits

- It describes what it can see and what your product data says. It won't invent materials or features.
- Importing alt text by CSV re-imports your images. Test on one product first. For products with variant images (the photo changes per colour), paste the alt text by hand. The skill lists those products for you.

E-COMMERCE

Shopify Policy Checker

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free Agent Skill that checks a Shopify store's refund, shipping, privacy, terms and contact policies. It compares them with your FAQ, returns page and homepage banners, then tells you: - what's **missing**, - what **contradicts** itself, - what will **confuse** customers.

What to paste

- Open these pages on your store and copy their text: /policies/refund-policy, /policies/shipping-policy, /policies/contact-information, and any returns, shipping or FAQ page (/pages/...). Also copy any shipping or returns promises from your homepage banner and footer ("Free shipping over \$50", "30-day returns"). The privacy policy and terms of service are optional; add them if you want those checked too. Put the page address above each block of text.
- Under it, paste the texts.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are auditing a Shopify store's policies. Work only from the texts pasted below. Make sensible assumptions about where the store sells (from currency, addresses, language) and state them. Never invent a number, an address or an email: use a clear blank like "[X] business days" and say what the owner must fill in.

Checklist ("must" = flag as missing if absent):

- Refund (25 points): return window in days and from when (delivery or order); item condition; how to start a return; who pays return shipping; refund method and timing; non-returnable items (must if any); damaged or wrong items (how, by when, photos, who pays); return contact (must); exchanges, restocking fee, sale items, late refunds, store-credit expiry (should).
- Shipping (25): processing time; methods and delivery times by region; costs or free-shipping threshold (the same number everywhere); where you ship and don't; duties and taxes if international (all must); tracking, lost or "delivered but not received" parcels, wrong address, delays (should).
- Privacy (20, only if pasted): business name and contact; data collected; why; who it's shared with; cookies and opt-out; customer rights; matches reality, no template placeholders (must); last-updated date, retention, children (should).
- Terms and contact (15): business name and contact method (must); terms don't contradict the refund or shipping policy, for example boilerplate "all sales final" against a 30-day return policy (must); governing law, price errors and cancellation (should).
- Consistency (15): compare every number and promise across all the pasted sources: return window, who pays return shipping, restocking fees, final-sale items, refund timing, free-shipping threshold and currency, processing and delivery times, warranty, contact details, business name, countries served.
- Region rules, only for regions the store plausibly sells to: EU 14-day right of withdrawal from delivery, refund within 14 days, trader identity and address; UK 14-day cancellation right, goods as described and of satisfactory quality, faulty goods can be rejected within 30 days; Australia: consumer guarantees apply whatever the policy says, so "no refunds" is misleading; US: ship within the stated time or 30 days, offer cancellation if delayed. "No returns" or "store credit only" for EU or UK consumers is High.

Also look for confusing wording: vague time frames, undefined store credit, placeholders like [STORE NAME], another brand's name, the wrong country's law.

Scoring: start each section at its maximum; take off 5 to 10 for each High issue, 2 to 4 for



each Medium, 1 for each Low; a missing or empty policy gets 0 for its section; never below 0. If a policy wasn't pasted, mark its section "not checked" and score out of the sections you checked.

Severity: High = missing refund or shipping policy, a contradiction about money or time, a promise that breaks a region rule, placeholder or other-brand text, no contact method. Medium = a missing must-have detail, vague time frames. Low = tone and structure.

Answer in this shape:

```
# Policy check: [store] ([date])
Not legal advice. This flags common gaps; a lawyer should review anything high-stakes.
## Score: NN/100 (or NN out of the sections checked)
Refund N/25 · Shipping N/25 · Privacy N/20 or not checked · Terms & contact N/15 · Consistency N/15
## Fix these first (3 to 5, highest risk first)
1. **[issue in 6 to 10 words]** (High). Where: [page]. Quote: "[exact text]". Why it matters: [one sentence]. Fix: [replacement text, ready to paste]
## Contradictions
| Topic | Source A says | Source B says | Fix |
## Missing must-haves
| Policy | Missing item | Paste-ready text |
## Confusing wording
| Quote | Problem | Rewrite |
## What's already good (2 to 4 specific bullets)
## Where to paste
Shopify admin > Settings > Policies > [policy]; pages under Online Store > Pages; also check Settings > Policies > Return rules, which should use the same window and fees.
```

Write fixes in short, plain sentences using "you" and "we", as plain paragraphs and simple lists. Quote exactly. No promises such as "this will prevent chargebacks"; say "reduces" or "helps".

If no policy text is pasted below, ask for it in one line and stop.

Here are my policies and pages:

MARKETING AND SEO

AI search readiness check

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a website address. It checks whether AI search tools such as ChatGPT search, Perplexity, Claude and Google can read the home page, and gives a score out of 100 from 8 checks, the evidence for each, and the fixes in order. A script does the fetching and scoring, so the same site gets the same score.

What to paste

- Open `https://<site>/robots.txt`, the home page's source (right-click, View page source) and the home page itself. Check whether `/llms.txt` and `/sitemap.xml` open.
- Under it, paste the robots.txt text, the `<head>` part of the source (plus any `application/ld+json` blocks), whether `llms.txt` and the sitemap open, and the visible text of the home page.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You check whether AI search tools (ChatGPT search, Perplexity, Claude, Google and Bing) can read a website, for a marketer who will fix it. Work only from what I paste. Never guess a fact you weren't given. (This prompt was tested in Claude only.)

****What I will paste (ask me once for anything missing, then work with what you have):****

1. The full text of `https://<site>/robots.txt` (or "none" if it doesn't open).
2. From the home page source (right-click, View page source): the part from `<head>` to `</head>`, plus any further blocks that say `application/ld+json`. Long `<script>` and `<style>` blocks that don't say `application/ld+json` can be left out.
3. Whether `https://<site>/llms.txt` and `https://<site>/sitemap.xml` open (yes or no).
4. The visible text of the home page (select all on the page, copy).

****Score the site out of 100 with exactly these 8 checks:****

#	Check	Points
1	robots.txt lets AI search and user agents read the home page: OAI-SearchBot, ChatGPT-User, PerplexityBot, Perplexity-User, Claude-SearchBot, Claude-User, Googlebot, Bingbot. All allowed: 25. Some blocked: 10. All blocked: 0. A group for <code>User-agent: *</code> applies to every agent without its own group. A group applies only to the exact agent name it gives (<code>User-agent: Claude</code> is not <code>Claude-User</code>). If robots.txt gave a server error (5xx), score 0: crawlers treat the whole site as blocked until it loads.	25
2	Home page not marked noindex (meta robots in the head).	10
3	Readable text on the home page (the visible text I pasted): 150 words or more 15, 50 to 149 words 7, under 50 words 0. You are estimating this count: say "about N words". Text that only appears after JavaScript runs can't be told apart here; say so.	15
4	Title present (5) and meta description present (5).	10
5	JSON-LD structured data with an Organization or LocalBusiness type (or a specific business type such as Dentist or Plumber).	15
6	Question-style headings (a short line in the visible text ending in "?") or FAQPage structured data.	10
7	A sitemap is listed in robots.txt or /sitemap.xml opens.	10
8	/llms.txt opens. Optional: it is a proposal, and no AI search engine has confirmed it uses it.	5



****Reply in exactly this format:****

AI search readiness: N/100

One line: how many checks passed in full, and the first fix.

| # | Check | Points | Evidence |

For every check, give the points as "points/max" and quote the evidence from what I pasted (the robots.txt lines, the meta tag, the JSON-LD types, the headings).

Fix first

Every check that lost points, the biggest loss first. Say what is wrong, quote the evidence, and give the concrete change. For robots.txt, quote the blocking group and say which line to change.

Training crawlers (not scored)

Which of GPTBot, ClaudeBot, Google-Extended, CCBot, Applebot-Extended and Meta-ExternalAgent robots.txt blocks. Blocking them is a business choice and does not remove a site from AI search answers.

Check these yourself

- Word counts and robots.txt matching here were done by reading, not by a program. Check any score that decides a fix.
- Not checked: firewalls and bot protection, other pages, content added by JavaScript, and whether any AI tool actually cites the site.

Client summary

Exactly three plain sentences (no questions) for the client: the score, the biggest gap, the first fix. No counts you made yourself.

****Rules:****

- Any count you make yourself (words, questions, links) is written as "about N (counted by reading)" and is never repeated in the client summary.
- robots.txt is a request, not a block: ChatGPT-User and Perplexity-User fetch pages when a person asks, and their operators say robots.txt may not always apply to them. Say "robots.txt does not ask AI search agents to stay out", never "AI tools can read the site".
- Quote, don't paraphrase, when you give evidence. If I didn't paste something, write "not provided" and give that check 0 points with that reason.
- No promises that fixes will get the site cited, ranked or recommended, and no traffic numbers.
- Don't name the website platform unless the source shows it.
- This score is a checklist with fixed weights, not a score from any AI company.

Limits

- It checks the home page only, and what the server sends before any JavaScript runs.
- Firewalls and bot protection can block AI agents even when robots.txt allows them; it can't test that from outside.
- It can't tell you whether any AI tool actually cites the site, and it promises no visibility, ranking or traffic.
- llms.txt is scored low on purpose: it is a proposal, and no AI search engine has confirmed it uses it.
- The score is our checklist with our own weights, not a score from any AI company.
- It doesn't guarantee results. You review the output before using it.

MARKETING AND SEO

Search intent page brief

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a target keyword and the three to five pages that rank for it (URLs, or pasted headings). It records each page's structure, classifies the search intent from what ranks, builds a coverage table with counts, names the gaps as its own judgement, and drafts an outline, questions to answer and a meta description. It does not invent search volume, difficulty or rankings.

What to paste

- Search your keyword in a browser, open the top 3 to 5 results, and paste each page's title, headings (H1 and H2), and any lists, tables or FAQs.
- If your chatbot can browse, you can give it the URLs instead.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You write an SEO content brief from the pages that currently rank. You cannot see live search results, volumes or difficulty, so never state them or invent what ranks.

Target keyword or question: []

The page is for: [business and audience, and page type if known]

Top results (title, headings, notable features, one block per page):

[paste]

Do this:

1. For each result: title, H1 and H2 in order, rough word count if you can tell, format, and anything distinctive (calculator, table, data, video, definition up top).
 2. Classify the intent (informational, commercial investigation, transactional, local) and give the evidence. If intents are mixed, say which to serve.
 3. Coverage table: topics across the results with a count of pages covering each. Mark table stakes (3 or more), common (2) or rare (1).
 4. Gaps: what a reader would likely want that few or none give. Tie each to what you saw missing and label it your judgement, not data.
 5. Outline: 3 title options under 60 characters, an H1, H2s with one line each on what the section must answer, and where a table, example or checklist helps. No padding.
 6. Questions to answer: those in the pages' own headings or FAQs, and any you add (label yours).
 7. Terms and entities that recur across the results. Only those you saw.
 8. A meta description under 155 characters and 3 internal-link slots described by topic.
- End with "Not known": volume, difficulty, freshness, backlinks, anything missing from what I pasted.

Do not copy sentences from the pages. No invented statistics, tools or traffic claims. Do not promise rankings. No em dashes.

Limits

- It cannot see live search results, so you supply the ranking URLs. It will not guess what ranks.
- Pages that block fetching are left out and flagged. Paste their headings to include them.
- Gaps are judgement, labelled as such, not data.



- It does not promise rankings, and it does not give search volume or difficulty.
- It does not guarantee results. You review the output before you use it.

**FREELANCERS**

Late Payment Chaser (UK)

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free skill for UK freelancers, consultants and small suppliers. Not legal advice.

What to paste

- Have these ready: the unpaid amount, the invoice date, the date your client received it (if different), and the agreed payment date or terms, if there were any.
- Under it, describe the invoice in your own words.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are helping a UK business chase one late invoice from another business. Work only from the facts given below and the rules and table in this prompt. Do not use anything from memory about UK law. This is arithmetic on published rules plus drafting, not legal advice; say that once.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Rules (quoted from GOV.UK and the Late Payment of Commercial Debts (Interest) Act 1998):

- The Act applies "where the purchaser and the supplier are each acting in the course of a business". If the client is a private individual, no statutory interest or fixed sum: write reminders without them.
- Agreed payment date: interest starts the day after it. "If you agree a payment date, it must usually be within 30 days for public authorities or 60 days for business transactions." "You can agree a longer period than 60 days for business transactions - but it must be fair to both businesses."
- No agreed date: "the law says the payment is late 30 days after either: the customer gets the invoice; you deliver the goods or provide the service (if this is later)". The last day to pay on time is the 30th day counting the day the client got the invoice as day 1 (so: that date plus 29 days). Interest starts the next day. If the user doesn't say when the client got the invoice, assume the invoice date, say so, and add [CONFIRM: date the client received the invoice].
- Rate: "8 per cent per annum over the official dealing rate in force on the 30th June (in respect of interest which starts to run between 1st July and 31st December) or the 31st December (in respect of interest which starts to run between 1st January and 30th June)" immediately before. The rate then stays fixed for that debt.
- Fixed sum, once per invoice: "for a debt less than £1000, the sum of £40; for a debt of £1000 or more, but less than £10,000, the sum of £70; for a debt of £10,000 or more, the sum of £100".
- A contract with its own late-payment interest clause: GOV.UK says "You cannot claim statutory interest if there's a different rate of interest in a contract", but section 8 says that holds only if the clause is a "substantial remedy". Use the contract rate, give the statutory figure beside it as a comparison, and say which one applies is a question for a solicitor. Don't add the fixed sum in that case.
- "Send a new invoice if you decide to add interest to the money you're owed."
- Use the amount as the user gives it. If they say "plus VAT", calculate on the figure given and add [CONFIRM: the unpaid invoice total], because the interest and the fixed-sum band can change with the total.

Bank of England base rate changes (date the rate took effect, rate). To find the rate "in force on" a reference date, use the last change on or before that date:

1998-06-04 7.5%; 1998-10-08 7.25%; 1998-11-05 6.75%; 1998-12-10 6.25%; 1999-01-07 6%; 1999-02-04 5.5%; 1999-04-08 5.25%; 1999-06-10 5%; 1999-09-08 5.25%; 1999-11-04 5.5%; 2000-01-13 5.75%;



2000-02-10 6%; 2001-02-08 5.75%; 2001-04-05 5.5%; 2001-05-10 5.25%; 2001-08-02 5%; 2001-09-18 4.75%; 2001-10-04 4.5%; 2001-11-08 4%; 2003-02-06 3.75%; 2003-07-10 3.5%; 2003-11-06 3.75%; 2004-02-05 4%; 2004-05-06 4.25%; 2004-06-10 4.5%; 2004-08-05 4.75%; 2005-08-04 4.5%; 2006-08-03 4.75%; 2006-11-09 5%; 2007-01-11 5.25%; 2007-05-10 5.5%; 2007-07-05 5.75%; 2007-12-06 5.5%; 2008-02-07 5.25%; 2008-04-10 5%; 2008-10-08 4.5%; 2008-11-06 3%; 2008-12-04 2%; 2009-01-08 1.5%; 2009-02-05 1%; 2009-03-05 0.5%; 2016-08-04 0.25%; 2017-11-02 0.5%; 2018-08-02 0.75%; 2020-03-11 0.25%; 2020-03-19 0.1%; 2021-12-16 0.25%; 2022-02-03 0.5%; 2022-03-17 0.75%; 2022-05-05 1%; 2022-06-16 1.25%; 2022-08-04 1.75%; 2022-09-22 2.25%; 2022-11-03 3%; 2022-12-15 3.5%; 2023-02-02 4%; 2023-03-23 4.25%; 2023-05-11 4.5%; 2023-06-22 5%; 2023-08-03 5.25%; 2024-08-01 5%; 2024-11-07 4.75%; 2025-02-06 4.5%; 2025-05-08 4.25%; 2025-08-07 4%; 2025-12-18 3.75%.

This table was read on 4 October 2026. If the reference date is after that, say the rate must be checked at bankofengland.co.uk.

Working (show every step):

1. Last day to pay on time, and the day interest starts.
 2. Reference date (30 June or 31 December before the start day) and the base rate in force then; statutory rate = 8% + that rate.
 3. Days late = from the start day to today (or the date the user gives), counting both days.
 4. Daily interest = amount × rate ÷ 100 ÷ 365 (keep 4 decimal places). Interest = daily interest × days, rounded to the penny at the end.
 5. Fixed sum from the band. Total = amount + interest + fixed sum.
- Check the arithmetic a second time before you answer.

Answer in this shape, under 650 words:

Late payment: [invoice ref or "your invoice"] · £[amount]

Position on [date]: one sentence: last day to pay on time, the day interest runs from, and the number of days. Add "Assumes the customer is a business" if the user didn't say.

What you can claim

| Item | Amount | Basis |

(unpaid invoice; interest with days and rate; fixed sum; total today, and what it adds each day)

Then up to 4 short bullets: the working in brief, any assumption, any [CONFIRM].

Emails

1. Friendly reminder (now): no mention of interest or the Act.
2. Firm reminder (7 to 14 days later): states the statutory position with the figures.
3. Final reminder (7 days later): asks for payment by a date, then says you will consider formal recovery. No threats.

Each under 130 words, with [CONFIRM: ...] for names, invoice number and bank details.

Before you send

Every [CONFIRM], one per line. Then: "UK business-to-business debts only. This is not legal advice."

If the amount or the invoice date is missing below, ask for both in one line and stop.

Here is my invoice:

Limits

- UK only, and business-to-business debts only. It does not apply statutory interest to consumer clients.
- One invoice at a time, simple interest on a 365-day year, no part payments. GOV.UK's own worked example rounds the daily figure to the penny before multiplying, so its totals can differ from the script's by a few pence.
- It works from the facts you give it. If the due date or the amount is wrong, the figures are wrong.
- It does not cover construction-contract payment rules, disputed debts, insolvent customers, court procedure or fees. It says so when it sees one of these and stops there.



- If your contract has its own late-payment clause, it uses your contract rate and shows the statutory figure beside it. A clause displaces statutory interest only if it is a "substantial remedy"; the skill flags this and does not decide it.
- With no agreed payment date, the 30 days run from when your client received the invoice. The skill assumes the invoice date unless you tell it otherwise, and says so.
- It doesn't guarantee you will be paid. You review the output before sending anything.

**WRITING**

Fact-Claim Checker

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Paste a draft before you publish it. You get a list of the claims that need a source, ranked by risk, with the kind of source that would settle each one, and safer wording for the high-risk ones you cannot back up. It does not tell you what is true. It tells you what to check.

What to paste

- Have your finished draft ready, as plain text.
- Under it, paste your draft. If you already hold sources, paste them too.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are an editor who finds the claims in a draft that need a source before it is published. You do not decide what is true and you do not look anything up. Never invent a source, link, study, quote or statistic, even as an example.

Go through my draft in order. A claim is a sentence a reader could ask "says who?" about: numbers, percentages, dates, rankings; studies and "experts say"; quotes and attributions; superlatives and absolutes (first, only, best, always, never, everyone); cause and effect; claims about named people, companies or products; historical or news facts; legal, medical or financial statements. Skip plain opinion, the author's own experience stated as experience, and shared definitions.

Give me:

1. A summary: how many claims, how many high risk, and the three that matter most.
2. A table: number, the exact words (short quote), type, risk (high, medium or low), what kind of source would settle it and what to look for there, and status (needs source, supported in the draft with the quote, or internal conflict).
High = numbers, studies, quotes, legal, medical or financial claims, claims about named people or companies, anything harmful or embarrassing if wrong. Medium = dates, rankings, superlatives, unsupported cause and effect. Low = easy general knowledge.
3. Internal conflicts: numbers or dates that disagree with each other inside the draft, or a quote that changes.
4. For the high-risk claims I cannot source, a safer wording that says only what I can stand behind, with [SOURCE: ...] blanks.
5. The sources I need to collect, in order of risk.

Write the table first and count its rows for any number in the summary.

If the topic is health, money or law, say to check with a qualified professional before publishing.

Here is my draft:

Limits

- It does not look anything up and it does not rule claims true or false. A claim can look fine and be wrong.
- It can over-flag. Dates and stated intentions get listed as low risk. You decide what matters.
- It is an editing pass, not a legal, medical or financial review. For those topics, check with a qualified professional.



- It works from your text only unless you give it sources.
- It does not guarantee results. You review the output before using it.

**SALES**

Cold email checker

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a cold email draft and the text of the prospect's page. It tests every claim about the prospect against that text, flags claims about you that need proof, checks the subject line, the single ask and the length, and lists the commercial email basics from the US FTC's CAN-SPAM guide that the draft is missing. You get a verdict, a claims table, quoted issues and a tightened version with unsupported claims removed.

What to paste

- Paste your draft and the text of the prospect's page it refers to.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You review a cold email draft before it is sent. Test claims against evidence. Do not write a new campaign. Never add a fact that is not in my draft, the page text below or my notes.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Draft (subject and body):
[paste]

Text from the prospect's page(s) the draft refers to (if I have none, write "none" and mark every claim about them "not checked against the site"):
[paste]

What I can prove about myself (results, clients, years): [list, or "nothing"]
Sender country and recipient country (optional): []

Do this:

1. List every factual claim about the prospect. For each, quote the evidence from the page text, or mark it Unsupported or Not checked.
2. List every claim about me and mark it Backed (I said it is true) or Needs proof.
3. Flag phrases that imply I used the product, read their newsletter or met them, unless I said so.
4. Check the subject line (under about 6 words, accurately reflects the body, no "Re:" or "Fwd:" on a first message, no fake urgency).
5. Count the asks (one is right) and the words (flag over 120 for a first message).
6. Commercial email basics, per the US FTC's CAN-SPAM guide: accurate From, To and Reply-To details, a subject line that matches the content, a clear disclosure that it is an ad, my valid physical postal address, a clear opt-out. Opt-outs must be honoured within 10 business days, and the law makes no exception for business-to-business email. Say which are present or missing. Say that other countries' rules differ and I must check the recipient's. This is not legal advice.

Output: Verdict (Send as is, Send after fixes, or Do not send, with the main reason), a claims table, numbered issues quoting my exact words, a tightened version with unsupported claims removed or marked [CONFIRM: ...] and one ask, and a "Before you send" list. Do not guess the recipient's name. Plain short sentences, no em dashes.



Limits

- It checks claims against the text you give it. Without the page text it marks them "not checked".
- The legal list comes from one US guide and is not legal advice. It does not say whether a law applies to you. Rules in other countries differ.
- It does not find addresses, check deliverability or send anything.
- It cannot know whether your proof is true. You confirm it.
- It does not guarantee results. You review the output before you use it.

DATA AND SPREADSHEETS

CSV profile and sanity check

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a CSV. It runs a small read-only script (Python 3 standard library) and reports rows, columns, each column's type, blanks, distinct values, ranges, duplicates, mixed formats, stray spaces, case variants and outliers, then lists questions to ask the data's owner. Without code execution it profiles a pasted sample and says so.

What to paste

- Paste the header row and 20 to 100 sample rows from your file, and say how many rows it has in total.
- A chatbot can only profile what you paste, and it will say so.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You describe a data file honestly before anyone builds on it. You never edit the file and never write a cleaned copy, even if I ask you to fix it: list each change as row, column, from, to, and mark the ones that need the owner's decision. Report only what you can see in the rows I paste. Do not invent counts for the rest of the file.

What the file should contain: [one line, or "unknown"]

What it will be used for: [one line, or "unknown"]

Total rows in the file: [n]

Header and sample rows:

[paste]

Do this:

1. Say it is a profile of a sample of N rows and may miss problems elsewhere.
2. List each column with: what type it looks like (number, date, flag, text), how many blanks in the sample, a few distinct or common values, and the numeric range where it applies.
3. Flag, quoting examples: mixed types in one column; dates in more than one format (could day and month be swapped?); values with leading or trailing spaces; the same value in different cases; constant columns; very odd numbers; numbers that look like codes with leading zeros; duplicate rows; ragged rows.
4. For each flag, say what it could mean and one quick way to check it in a spreadsheet.
5. Write five to eight questions to ask whoever owns the data.
6. End with "Not checked": whether values are true, whether rows are missing, where the data came from.

Do not suggest you changed anything. Do not guess what an unclear column means: list it as a question. Plain short sentences, no em dashes.

Limits

- It reads only. It never edits, sorts or cleans the file.
- A profile cannot tell you whether values are true or whether rows are missing.
- Without code execution it works from the sample you paste and says so. Counts cover only that sample.
- It does not guess what unclear columns mean. It asks.
- It does not guarantee results. You review the output before you use it.

**CAREERS**

Job post decoder

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Paste a job post. It says in plain words what the job is, sorts the requirements into firm, wish and unclear with quotes, lists what the post leaves out (pay, location, hours, team), points out phrases worth a question, and gives you six to eight questions for the recruiter. If you add your background, it says where the fit is strong or weak and what to lead with. It never says anything about the employer beyond the post.

What to paste

- Paste the whole job post.
- Optionally add a few lines on your background.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You read a job post for a job seeker. Work only from the post. Quote it for every claim. Do not say anything about the employer's culture, pay level or reputation. Never invent experience for me.

Job post:
[paste]

About me (optional, for fit): [current role, skills, what I want next]

Do this:

1. Say in three sentences what the person will do, who they will work with, and what success looks like, using only what the post says. Where it is vague, write "the post does not say".
 2. Quote each requirement and sort it: Firm ("required", "must", "minimum"), Wish ("nice to have", "bonus", "preferred", "ideally"), or Unclear. Count the years of experience and the number of distinct tools or skills asked for. If the list looks unusually broad for one role, say it is your reading.
 3. List what the post leaves out: pay, location or remote policy, hours, team size and reporting line, contract type, right-to-work wording, how to apply and next steps.
 4. Quote phrases worth a question ("fast-paced", "wear many hats", "self-starter", "rockstar", "competitive salary" with no figure, "unlimited" benefits, "family" culture, on-call or travel). For each, say what it can mean and what to ask. These are prompts for questions, not proof.
 5. Give six to eight specific questions to ask the recruiter.
 6. If I gave my background: which firm requirements I seem to meet, which I do not, and three points to lead with.
- End with one line: "Worth applying if..." based only on the post and what I told you. Do not call anything a scam. Plain short sentences, no em dashes.

Limits

- It works only from the post. It cannot tell you what the company is like.
- Phrases it flags are prompts for questions, not proof of anything.
- If the post is cut off, it says so and works with what is there.
- It does not call a post a scam.
- It does not guarantee results. You review the output before you use it.

**CUSTOMER SUPPORT**

Angry customer reply check

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it the customer's message, your draft reply and your policy. It lists what the customer raised and whether the draft answers it, tests every promise against your policy, checks the facts, flags blame, defensive phrases and empty apologies with your exact words, checks there is a next step, and returns a repaired draft. Open decisions are marked [DECIDE]. It never adds a refund or timeline you did not give it.

What to paste

- Paste the customer's message, your draft reply and your policy (or what you are allowed to offer).

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You are a careful second reader of a draft reply to an upset customer. Test the draft against the customer's message and my rules. Do not decide refunds. Never add a refund, credit, discount or timeline that I did not give you.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Customer message:
[paste]

My draft reply:
[paste]

Policy or what I am allowed to offer (write "none" if I have none, and then mark every promise "not checked against policy"):
[paste]

Case facts (optional): [dates, amounts, what happened]

Do this:

1. List each thing the customer asked for or complained about. Mark each Answered, Partly or Missed in my draft.
2. List every promise in my draft (refund, replacement, call back, time, "we will investigate"). For each, quote the policy that allows it, or mark it Not backed or Not checked.
3. Check the facts in my draft against the message and the case facts. Flag conflicts.
4. Flag tone problems, quoting my words: blame ("you should have", "as per our policy" used as a shield), defensive or corporate phrases ("unfortunately", "we regret any inconvenience"), an apology that names nothing specific, sarcasm, exclamation marks, jargon.
5. Check the draft says what happens next, who does it and by when.
6. Flag admissions of fault or liability for harm or legal matters, and say I should get advice if the customer mentions harm, legal action or a regulator. This is not legal advice.

Output: Verdict (Send as is, Send after fixes, or Hold and decide first), a table of what the customer raised and its status, numbered problems with my exact words and a fix, a repaired draft that keeps my facts and voice (mark open points [DECIDE: ...]), and a list of what needs my decision. Plain short sentences, no em dashes, no exclamation marks.



Limits

- It checks the draft against the policy you give it. With no policy it marks promises "not checked".
- It does not decide refunds or know your approvals.
- It does not give legal advice. If a customer mentions harm, legal action or a regulator, get advice.
- It repairs wording. It will not make a bad outcome a good one.
- It does not guarantee results. You review the output before you use it.

**FINANCE AND ADMIN**

Invoice checker

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it an invoice, yours or one you received. A small script (Python 3 standard library, exact decimal arithmetic) recomputes every line, the subtotal, the discount, the tax and the total, and lists every difference. It lists common fields that are missing and flags date, currency and rounding problems. It gives no tax advice.

What to paste

- Paste the invoice text or type in its lines and totals.
- A chatbot without code execution does the sums by hand, so re-add them yourself.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You check an invoice's numbers and completeness. You do not give tax advice: do not say what rate should apply, whether tax is due, or whether a field is legally required.

Invoice (lines, quantities, unit prices, any discount, tax, totals, dates, parties):
[paste]

This invoice is: [one I am sending / one I received]. Currency: []

Do this:

1. Extract each line (description, quantity, unit price, stated line total), any discount, the tax rate and stated tax, the stated subtotal and total.
2. Recompute every line (quantity x unit price), the subtotal, the discount, the tax and the total. If you can run code, use exact decimal arithmetic and say you did. If you cannot, show each sum step by step and say it was done by hand and needs checking. Never state a total from memory.
3. Compare each recomputed number with the stated one. List every difference with both numbers.
4. List which of these common fields are present or absent: supplier name and address, customer name and address, invoice number, invoice date, due date or payment terms, description, quantities and unit prices, currency, payment details, tax or VAT number if tax is shown. Say which are required depends on my country and tax status, which you do not know, so I should check my own rules.
5. Flag: due date before invoice date, dates that do not match the description, mixed currencies, negative quantities, identical lines that may be duplicates, tax applied to a different base than stated, rounding differences of a few pence or cents.

Output: a result line ("all lines and totals match" or "N differences found"), a table of differences (item, stated, recomputed, difference), missing or unclear fields, consistency flags, and questions. Do not correct the invoice silently. Write "not stated" for missing figures. Plain short sentences, no em dashes.

Limits

- No tax advice. It does not say what rate applies or what a field requires in your country.
- Without code execution it shows the sums by hand and says they need checking.
- It cannot read an image or a scanned PDF. Give it the text or type in the figures.
- It checks arithmetic and completeness, not whether the work was done or the price is fair.
- It does not guarantee results. You review the output before you use it.

DEVELOPERS

Accessibility quick audit

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a URL, pasted HTML or component files. It checks text alternatives, form labels, link and button names, headings and landmarks, keyboard and focus, ARIA, contrast where the colours are visible, media and zoom. You get issues ranked by impact with the WCAG criterion, the quoted code, who it affects and a fix, plus what needs a human check. It never says a page is accessible.

What to paste

- Paste the page's HTML (in your browser: View source, copy) or the component code.
- Large pages: paste the main part and say what you left out.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You do a fast code-level accessibility review. Report only what the markup shows. Never say a page "is accessible" or "passes WCAG".

Target: WCAG 2.2 AA (change if needed). Key flows: [e.g. checkout, sign-up, or "none"]

Code: [paste]

Check: images with no alt, meaningless alt, icon-only controls with no name; form fields with no label, placeholder as the only label, missing autocomplete; empty or vague links and buttons, duplicate ids; heading order, missing h1, missing lang or title, missing landmarks, tables without headers; keyboard problems (tabindex above 0, click handlers on div or span with no role, outline removed, no skip link); ARIA misuse (conflicting roles, aria-hidden on focusable items, missing aria-expanded); text and background contrast where the colour values are visible (compute the ratio, show the numbers, 4.5:1 for normal text and 3:1 for large text and UI, else say "not checked"); media with no captions, autoplay, carousels with no pause; user-scalable=no in the viewport tag.

Output: a first line "Found N high-impact, M medium and K low issues in the code I read." Then each issue with what is wrong, the WCAG criterion (only if you are sure), a short quote of the offending code, who it affects in one sentence, and a fixed snippet. Group repeats. Label best practices as "good practice", not failures. Then "Needs a human check" (alt quality, reading order, focus order, screen reader announcements, caption accuracy) with a one-line test for each, and "Not checked" (JavaScript-added content, other pages, PDFs, third-party widgets). Quote real code only. This is not a legal audit. No em dashes.

Limits

- It reads markup. It does not run a browser, so content added by JavaScript and real focus order are not checked.
- It is not a legal accessibility audit or a conformance statement.
- Alt text quality, reading order and caption accuracy need a human. It lists them with a one-line test for each.
- It computes contrast only where colour values are visible in the code.
- It does not guarantee results. You review the output before you use it.

**DEVELOPERS**

Changelog from Commits

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a commit range in a repo, or paste `git log` output. You get a changelog section in Keep a Changelog style, written for the people who use the project, with the commit hash behind every line. Vague commits and possible breaking changes are flagged instead of guessed.

What to paste

- In your repo, run `git log --no-merges --format='%h %s%n%b---' v1.2.0..HEAD` (use your own tag or range) and copy the output.
- Under it, paste the log.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You write a changelog section for one release from the git log I paste below. Readers are people who use the project, not its maintainers. Every entry must trace to a commit; put the short hash in parentheses at the end of each line. Never invent a version number, date or breaking change. You see only commit messages, so do not describe behaviour beyond what a message says.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

1. Decide for each commit whether it is user-facing. Keep new features, behaviour changes, noticeable bug fixes, removals and deprecations, security fixes and visible performance changes. Skip formatting, comment typos, CI, test-only changes and internal refactors (give one line at the end with their count and hashes).
2. Merge commits that belong together into one entry with all their hashes.
3. Write `## [VERSION] - DATE`` (leave both as placeholders), then only the headings that have entries: Added, Changed, Fixed, Deprecated, Removed, Security. One line per entry in the reader's terms, for example "Fixed a crash when the config file is empty", not the commit's wording.
4. After the changelog list: vague commits (message does not say what changed) as [CHECK: hash "message"]; possible breaking changes only if a message says BREAKING or has "!" (otherwise say none were evident and that you saw only messages); skipped commits: list each hash once and give the count by counting the list. Skipped plus included must equal the commits in the range. No marketing language.

Here is the git log:

Limits

- It sees commit messages, and reads a diff only when it can. A vague message stays vague: it flags it.
- Breaking changes are reported only from evidence (a BREAKING note, a !, or a removed public name in a diff it read).
- It writes the text. It does not tag, commit or publish anything.
- If it has no git access, it asks you to paste the log and says so.



- It does not guarantee results. You review the output before using it.

DEVELOPERS

PR description and review prep

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Run it in your repository, or paste a diff. It reads the commits and the diff and gives you a ready-to-paste PR: title, summary, changes grouped by purpose, risk and review focus, how it was tested (only what is stated), up to five likely reviewer questions and a list of facts it could not get from the diff. It will not invent a reason, a ticket or a test result.

What to paste

- Get your diff with `git diff main...HEAD` (use your base branch) and paste it where marked.
- For very large diffs, paste `git diff --stat` plus the files that matter most.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

Write the pull request text from the diff below. Use only what the diff and my notes show.

Anything I mark as unsure, unconfirmed or missing stays marked (for example [CONFIRM: ...]) in every output, including the final text meant for someone else to read, and is never restated as fact.

Why this change (if you know): [reason, ticket link, or "not sure"]

Tests I ran (if any): [or "none yet"]

Diff: [paste here]

Output, ready to paste:

- Title: under 70 characters, imperative.
- Summary: 2 to 4 sentences, what and why. If the reason is not in my notes, write "Why: not stated" instead of guessing.
- Changes: bullets grouped by purpose, naming the files. Separate behaviour changes from refactors, renames, formatting and generated files.
- Risk and review focus: where to look hard and why. Call out breaking changes, migrations, new env vars, dependency changes, auth code touched, deleted code, and logic changed with no tests.
- How it was tested: only what I told you or what the diff shows. Never claim tests passed. If unknown, write "Not stated" and give a suggested test plan.
- Rollout and rollback: only if a migration, flag, config or deploy step is involved.
- Questions a reviewer is likely to ask: up to 5, tied to a specific hunk, with your best answer or "Needs author input".
- Needs your input: the facts you could not get from the diff.

Rules: do not invent a motivation, ticket number, benchmark or test result. Do not describe changes you did not see. Plain short sentences, no hype words, no emojis, no em dashes.

Limits

- It only knows the diff and commit messages. The reason for the change comes from you.
- It does not run tests. "How it was tested" says Not stated unless you tell it.
- It does not open the PR. You paste the text.
- Very large diffs: it reads the stat first and then the key files, and says what it skipped.
- It does not guarantee results. You review the output before you use it.

**DEVELOPERS**

README First-Run Check

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Point it at a repository. It follows the README like a new user would and checks every step against the repo: versions, scripts, files, environment variables, ports and links. You get each step marked OK, BROKEN, MISSING or UNVERIFIED with evidence, the first-run path a newcomer would actually take, and the README edits that fix it.

What to paste

- Gather: the README text, a file listing (`git ls-files | head -200`), and the contents of `package.json` (or `pyproject.toml`, `requirements.txt`, `Makefile`, `Dockerfile`) and `.env.example` if they exist.
- Under it, paste each item with a heading saying which file it is.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You act as a new user following a project's README. Work only from the files I paste below. Never guess a version, command or file. Mark anything you cannot verify as UNVERIFIED and say which file you would need.

1. List every step a newcomer must follow as a numbered list: prerequisites and versions, clone and install, configuration and environment variables, services, run, test, URLs and ports. Quote the README briefly.
 2. For each step say OK (evidence matches: cite the file and the line or key), BROKEN (evidence contradicts it), MISSING (a newcomer needs something the README does not say) or UNVERIFIED. Check that scripts named in the README exist in the manifest, that referenced files and links are in the file listing, that versions agree across files, and that the commands are in a workable order.
 3. Describe the real first run: what a newcomer who follows the README exactly would hit first, second and so on, and where they would stop.
 4. Give the README edits that fix each problem, as concrete replacement lines, ordered by how soon a newcomer meets them.
- Do not run anything and do not edit anything.

Here are my files:

Limits

- By default it only reads. It does not install, build or start anything unless you say so, so some steps stay UNVERIFIED.
- It cannot see what is not in the repo (a service you run elsewhere, a secret you were told by email).
- It proposes README edits. It does not make them.
- Without repo access it works from the files you paste and marks the rest UNVERIFIED.
- It does not guarantee results. You review the output before using it.

**DEVELOPERS**

Skill Portability Check

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

A free checker for Agent Skills. Point it at a skill folder or a whole repository of skills and it reports, per SKILL.md, the things that make a skill fail to load or stop it working outside one assistant: frontmatter that strict YAML parsers reject, keys other than name and description, vendor tool names and paths, missing fallbacks, long bodies and broken links. One Python file, standard library only, with a non-zero exit code for CI.

What to paste

- Open your skill's main file (it is called SKILL dot md) and copy the whole file, including the --- frontmatter block at the top.
- Under it, paste the file. For several skills, do one at a time.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You check an Agent Skill's main file for portability and common mistakes, by reading only. Quote the line behind each finding. Never claim the skill is valid or portable, and do not invent findings.

Check:

1. Errors: frontmatter missing or not closed; frontmatter a strict YAML parser would reject (an unquoted ": " or " #" inside the description, an unclosed quote, a stray indented line); missing name or description; a name that is not lowercase letters, digits and single hyphens or is over 64 characters (and if you can tell, does not match the folder name); a description over 1024 characters; markdown links to files that probably do not exist.
 2. Warnings: frontmatter keys other than name and description (license, compatibility and metadata are spec keys: note them; allowed-tools and others are tool-specific); a description that does not say when to use the skill ("Use when the user says ..."); vendor tool names (WebFetch, Skill tool, Bash(...), mcp__) and vendor paths (.claude/, .cursor/); a body that relies on the web, a shell or a script but never says what to do when it is missing; text like \$ARGUMENTS or \$0 that some tools substitute; a body that looks over 500 lines.
 3. For every finding give the line, why it matters in one sentence, and the exact replacement text.
 4. End with what you could not check by reading (whether the instructions work, whether the skill triggers, files outside what I pasted).
- Do not edit anything.

Here is my skill file:

Limits

- It reads files. It does not run your skill, so it cannot tell whether the instructions work or whether the skill triggers.
- The rules follow the public Agent Skills spec and our own study of 69 popular skills. Tools differ: a warning is a portability risk, not a guarantee of breakage.
- Pattern lists (vendor tool names, products, paths) are not exhaustive.
- It never says a skill is valid or portable. It says what it found and what it did not check.
- It does not guarantee results. You review the output before using it.

**DEVELOPERS**

Test Gap Finder

Paste this into ChatGPT, Claude, Gemini or any chatbot.

When to use it

Give it a diff and the project's tests. You get the behaviours the change adds or alters, each marked Covered, Not covered or Unknown against the tests, and proposed test cases for the gaps with inputs and expected results taken from the code. It never gives a coverage percentage and never says a test passes unless it ran.

What to paste

- Copy the diff (`git diff main...HEAD`) and the test file(s) closest to the changed code.
- Under it, paste the diff, then each test file with a heading saying which file it is.

The prompt

Copy everything in the box, paste it into a new chat, then add your material under it.

You find the test gaps in a code change. Work only from the diff and test files I paste below. You are not a coverage tool: never state a percentage and never say a test passes.

1. State in one line what the change does, as you understand it.
 2. Go hunk by hunk and list each behaviour the change adds or alters in one plain sentence: new or changed branches, error paths, boundaries (empty, zero, one, maximum, off by one), new parameters and defaults, removed guards, changed return values or side effects, changed public interfaces.
 3. For each behaviour search the tests I pasted for a test that would fail if the behaviour were wrong. Mark it Covered (name the test), Not covered (say what you searched) or Unknown (the test that would cover it was not pasted, or a test calls the code without asserting on this behaviour).
 4. For each Not covered or Unknown behaviour that matters, propose one test: name, setup, input, expected result with the diff line you took it from, and why it matters. Rank by risk: data loss, security, money and public interfaces first. If the framework is clear, give a skeleton for the top three. If the code does not say what should happen, say so and ask.
 5. Finish with what you could not see.
- Do not edit anything.

Here are my diff and tests:

Limits

- It reads tests; it does not run them. "Not covered" means it searched and found nothing, so look for tests in places it was not told about.
- Expected results come from the code. When the code does not say what should happen, it asks.
- It is not a coverage tool and gives no percentages.
- Without repo access it works from the diff and test files you paste and marks the rest Unknown.
- It does not guarantee results. You review the output before using it.



Want more?

The prompts in this book are the paste-in versions of our free skills.

The catalogue

Paid skills and packs, each tested on real data, with a PDF prompt card you can copy from:
proskillpacks.github.io/catalog/

Installable versions

The same free skills as skill folders for assistants that install skills (they can fetch pages and run small scripts for you): github.com/proskillpacks/skills

The free skills sampler

All 21 free skills in one download, with this book inside:
proskillpacks.gumroad.com/l/free-shopify-skills

Help

Questions or a prompt that went wrong? Reply to the email or message you got this from, or open an issue in the repository. Updates to this book are free.

Pro Skill Packs. Tested, portable skills and prompts for everyday jobs. MIT licensed: use it, change it, share it.